

Notes on Value Iteration

Luis Mendez

VALUE ITERATION

Value iteration is a dynamic programming algorithm for computing the optimal value function V^* (and from it, the optimal policy π^*) of a Markov Decision Process, when we already have a model of the environment (transition probabilities $P(s'|s, a)$ and reward function $R(s, a, s')$). It works by directly turning the Bellman optimality equation into an update rule and applying it repeatedly until the value function stops changing.

Bellman Optimality Equation

The optimal value function satisfies

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V^*(s')].$$

This equation says the value of a state under the optimal policy equals the value of the best action available from that state, where the best action is the one that maximizes immediate reward plus the discounted value of wherever we land next. It is not directly solvable in closed form because V^* appears on both sides, so instead we turn it into an iterative update.

The Update Rule

Starting from an arbitrary initial value function V_0 (commonly all zeros), at every iteration k we update every state simultaneously:

$$V_{k+1}(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V_k(s')], \quad \forall s.$$

Each sweep over all states is one iteration. Unlike policy iteration, value iteration does not maintain an explicit policy while it runs, it only tracks the value function, we recover the greedy policy only at the end (or whenever we need it) by taking

$$\pi(s) = \arg \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')].$$

This is the key difference from policy iteration: policy iteration alternates between a full policy evaluation pass (solving for V^π exactly, or nearly exactly) and a policy improvement pass, whereas value iteration can be seen as truncating the policy evaluation step to a single sweep, folded together with the improvement (the max) at every state, every iteration.

Algorithm

- 1) Initialize $V(s) = 0$ for all states s (or any arbitrary values), pick a small threshold $\theta > 0$.
- 2) Repeat:
 - a) $\Delta \leftarrow 0$

- b) For each state s :

- i) $v \leftarrow V(s)$
- ii) $V(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$
- iii) $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

- 3) Until $\Delta < \theta$ (the value function has essentially stopped changing).

- 4) Output the greedy policy π with respect to the final V .

In practice, states can be updated in place (using the newest values of $V(s')$ as soon as they are computed, rather than keeping a separate copy for the whole sweep) which tends to converge faster and still works because of how the Bellman backup is a contraction, described below.

Why it Converges

The Bellman optimality backup operator $\mathcal{T}$ defined by

$$(\mathcal{T}V)(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$

is a contraction mapping in the max-norm, with contraction factor equal to the discount factor $\gamma < 1$:

$$\|\mathcal{T}V_1 - \mathcal{T}V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty.$$

By the Banach fixed point theorem, repeatedly applying a contraction mapping converges to a unique fixed point regardless of the starting point, and that fixed point is exactly V^* (since V^* satisfies $\mathcal{T}V^* = V^*$ by definition of the Bellman optimality equation). This is why value iteration is guaranteed to converge to the optimal value function, and why the discount factor γ controls the convergence rate, a γ closer to 1 converges more slowly (in the undiscounted case, $\gamma = 1$, this contraction guarantee no longer holds in general).

Convergence in Practice

Value iteration in theory only converges in the limit as $k \rightarrow \infty$, in practice we stop early once $\Delta < \theta$ for a small threshold θ . This gives an approximately optimal value function, the resulting greedy policy is often already optimal well before V itself has fully converged, since the policy only depends on the ordering of the Q -values for each action, not their exact magnitude.

Value Iteration vs. Policy Iteration

- **Cost per iteration:** value iteration does a single Bellman backup per state per sweep ($O(|S|^2|A|)$ per sweep for a naive implementation), whereas policy iteration's evaluation step usually requires solving a system of linear equations or repeatedly sweeping until V^π converges, which is more expensive per iteration.

- **Number of iterations:** policy iteration tends to converge in fewer iterations (since it always evaluates the current policy close to exactly before improving it), but each of its iterations is more expensive. Value iteration needs more iterations but each is cheap.
- **What's tracked:** policy iteration explicitly maintains a policy π at every step, value iteration only tracks V and derives π at the end.
- In practice, for problems with a moderate number of states, value iteration is often simpler to implement and reason about, since it is a single loop with a single update rule.

Requirements

Like all dynamic programming methods for MDPs, value iteration assumes we have a full model of the environment (P and R are known), and that the state and action spaces are small enough to enumerate and sweep over exhaustively. When the model is not known, or the state space is too large, model-free methods (e.g., Q-learning, which can be seen as a sampled, asynchronous version of the same Bellman backup) are used instead.